

PHANTOM PROTOCOL



Security Architecture Audit

Complete Cross-Specification Vulnerability Assessment

March 2026 · Draft v0.1 · Confidential



Audit Scope

This report audits the complete Phantom Protocol architecture across all three technical specifications: Spec №1 (Hidden Services), Spec №2 (Sphinx⁺ and ZK Shuffle), and Spec №3 (DPKI/Kademlia DHT), as well as the v0.1 Whitepaper. Each finding is cross-referenced to the specific specification section and assigned a severity level. Research was conducted against the current state of the art in mix-network attacks, post-quantum cryptography vulnerabilities, distributed hash table security, zero-knowledge proof system weaknesses, and traffic analysis techniques as of March 2026.

Severity	Count	Key Finding Areas
● CRITICAL	2	Kyber fragment scheme (novel, unaudited construction); BLS12-381 trusted setup — no CRS ceremony plan exists
● HIGH	5	Intersection attack surface; DC-Net jamming; active Sybil on DHT; QUIC fingerprinting; clock skew epoch boundary
● MEDIUM	6	Batch timing oracle; proof-of-work GPU scaling; 8-byte C_batch truncation; DHT record poisoning; cover traffic model; PoW difficulty lag
○ LOW	4	Replay cache memory exhaustion; mDNS bootstrap privacy; SURB size side-channel; epoch-0 KEM keys in address
ℹ INFO	3	Missing formal threat model doc; no incentive mechanism; genesis CID upgrade path

Table 1 — Finding summary by severity

1. Executive Summary

Phantom Protocol presents a technically ambitious architecture that is more rigorous than any deployed mix network. The ZK shuffle proof, DHT-native PKI, and hybrid post-quantum key exchange represent genuine innovations. However, several structural weaknesses were identified during this audit that require resolution before the protocol can be considered production-ready against its stated GPA threat model.

The two most serious findings are:

- **CRIT-01:** The Kyber-1024 96-byte fragment scheme in Sphinx⁺ is a novel, unreviewed cryptographic construction with no security proof. The specification acknowledges this but does not provide a timeline or alternative. If this construction is broken, post-quantum secrecy on the entire mix network collapses to the classical X25519 layer.
- **CRIT-02:** No CRS (Common Reference String) generation ceremony for BLS12-381 is planned or described. The ZK shuffle proof's soundness — the core anonymity guarantee of the entire protocol — depends on this ceremony having been conducted correctly and the trapdoor destroyed. Without it, a single party could forge shuffle proofs and deanonymize all traffic silently.

Beyond these, five HIGH-severity findings address the intersection attack surface (which the protocol significantly underestimates), DC-Net jamming (which is underspecified), active Sybil attacks on the DHT using the IPFS-demonstrated technique of responding with semantically valid but false data, QUIC fingerprinting by ML-based DPI (which bypasses the 'mimic HTTPS' assumption), and epoch clock skew attacks that allow injection of expired descriptors across boundary windows.

None of these findings are fatal to the design. Each has a tractable mitigation. The recommendations in this report are ordered by implementation priority.

2. Audit Methodology

The audit was conducted in three passes:

1. **Specification consistency analysis:** All three technical specifications and the whitepaper were read in full and cross-referenced for internal contradictions, undefined behaviour, and underspecified failure modes.
2. **Threat surface mapping:** Each architectural component was mapped to its threat surface. Known attack classes from the literature were evaluated for applicability, including: traffic analysis and intersection attacks, DHT Sybil and eclipse attacks, ZK proof system weaknesses, side-channel attacks on post-quantum primitives, transport-layer fingerprinting, and protocol-specific attacks (batch timing, epoch boundary, cover traffic).
3. **State-of-the-art literature review:** Current (2024–2026) research on each relevant attack class was reviewed to assess whether the protocol's defences remain adequate against modern adversary capabilities. This included the IPFS active Sybil attack (Inria, May 2025), KyberSlash timing attacks and SPA variants (Bernstein et al., 2024; Berzati et al., 2026), ML-based QUIC traffic classification (ESORICS 2024, HotNets 2025), and MixMatch flow correlation for mixnets (PoPETs 2024).

⚠ Severity Definitions

CRITICAL: Breaks a core security guarantee (anonymity, authenticity, or forward secrecy) silently or at scale. Must be resolved before any public deployment.

HIGH: Significantly degrades security in realistic adversary scenarios. May not break guarantees entirely but substantially narrows the anonymity set or enables deanonymization under achievable conditions.

MEDIUM: Exploitable under specific conditions or by well-resourced adversaries. Degrades performance of security guarantees without eliminating them.

LOW: Minor issues that marginally affect security or operational robustness. Should be addressed before production but are not blocking.

INFO: Observations about design gaps that do not directly cause vulnerabilities but represent missing components or documentation.

3. Critical Findings

CRIT-01 Kyber Fragment Scheme: Novel Unaudited Construction

• **CRITICAL** Location: Spec №2 §3.4, §3.2.4

The 96-byte per-hop Kyber-1024 fragment scheme is a novel protocol construction with no published security proof. The specification acknowledges this but treats it as a lower-priority audit item. In practice, it is the single most significant unreviewed component in the entire stack.

Kyber-1024 (ML-KEM-1024) ciphertexts are 1568 bytes. Sphinx⁺ compresses these to 96 bytes per-hop using a custom 'fragment scheme' described only in one paragraph of the specification. The description claims that '96 bytes is sufficient for decapsulation given the node's private key and the sender's public key via the implicit rejection mechanism.' This claim is not proven.

Several concerns arise from this construction. First, Kyber's IND-CCA2 security depends on the Fujisaki–Okamoto transform, which requires that the full ciphertext be available during decapsulation. Truncating the ciphertext to 96 bytes and expecting the recipient to reconstruct the rest from context breaks the assumptions of the FO transform. Second, even if decapsulation succeeds, the unlinkability property — that successive α_{pq} values across hops are computationally indistinguishable — is asserted but not demonstrated.

This is compounded by recent research: KyberSlash (Bernstein et al., 2024) demonstrated secret-dependent timing leakage in Kyber decapsulation, and a 2026 paper (Berzati et al., CASCADE 2025) showed that even the patched implementations leak via power analysis on embedded hardware. While Phantom targets server-grade hardware where these attacks are harder, the combination of an unaudited fragmentation scheme and known side-channel sensitivity in the underlying primitive is a serious concern.



The Math Problem

A Kyber-1024 ciphertext encodes a polynomial-encoded key and an encrypted message. Truncating the encoding to 96 bytes loses polynomial coefficients. For the recipient to decapsulate, they would need to reconstruct the missing coefficients — which requires knowing values that are secret by design. The 'implicit rejection mechanism' in FIPS 203 handles decapsulation failure by returning a pseudorandom value, not by allowing partial ciphertext recovery.

→ Recommendations

- Treat the Kyber fragment scheme as unproven and engage a cryptographer to produce either a security reduction or a proof of insecurity before Phase 2 begins.
- As an interim measure, consider an alternative: pre-encapsulate all per-hop Kyber shared secrets at packet construction time (before entering the mix), storing each full ciphertext in a separate onion layer. This increases header size but uses the standard Kyber construction without truncation.
- Reference the Outfox paper (Nym, arXiv:2412.19937, 2024) which addresses post-quantum layered-mixnet packet formats — it may provide a viable construction that has received more scrutiny.

- Flag the X25519 layer as the primary security guarantee and explicitly document that the Kyber layer provides harvest-now-decrypt-later resistance only until the fragment scheme is audited.

CRIT-02 BLS12-381 Trusted Setup: No CRS Ceremony Plan

• **CRITICAL** Location: Spec №2 §4.2, §8.5, Whitepaper §12

The ZK shuffle proof requires a BLS12-381 Common Reference String (CRS) generated in a trusted setup ceremony. Without this ceremony completed before mainnet, no valid shuffle proofs can be generated — and if the ceremony is compromised, shuffle proofs can be forged silently, breaking the entire anonymity guarantee.

The Bayer-Groth verifiable shuffle argument used in the Phantom ZK Shuffle requires a CRS containing group generators ($g, h_1, \dots, h_k$) for BLS12-381 G1. The specification references this as 'CRS from network genesis' but provides no description of how this CRS is generated, verified, or distributed.

The trusted setup problem for Groth16-class systems is well understood: the CRS is generated by computing $[\tau]_1, [\tau^2]_1, \dots, [\tau^n]_1$ for a secret trapdoor τ . If any single party knows τ , they can forge proofs — in Phantom's case, this means forging shuffle proofs, which allows a malicious node to forward packets in any order (including input order) while producing a proof that passes verification. This silently eliminates the core anonymity guarantee of the mix network.

The specification acknowledges this as a known limitation and mentions a STARK migration roadmap, but makes no provision for conducting a ceremony before deployment or for using a universally structured reference string (like Zcash's Powers of Tau) that reduces trust assumptions through multi-party computation.

The Cheon attack (2015) further complicates large trusted setups: for setups generating $[\tau^d]_1$ where d is known, the security is reduced from discrete log hardness to approximately 2^{125} operations — not 2^{255} . For the Phantom batch size (N up to 512), the degree d is proportional to N , keeping this within acceptable bounds, but the attack analysis is absent from the specification.

→ Recommendations

- Plan and conduct a public MPC ceremony for the BLS12-381 CRS before mainnet launch. Use the 'Powers of Tau' construction so that the ceremony is useful for any future circuit changes, not circuit-specific. The ceremony requires at least one honest participant to destroy their partial trapdoor.
- Consider using an existing Powers of Tau transcript (e.g., from the Zcash Sapling ceremony or the Ethereum KZG ceremony) as a foundation, if the circuit size is compatible.
- Document the CRS parameters, the ceremony transcript, and the verification procedure in a separate security document so that any network participant can independently verify the CRS integrity.
- Accelerate the STARK migration roadmap to Phase 2 rather than Phase 5, since eliminating the trusted setup removes the highest-trust assumption in the entire

protocol. Candidate systems: Plonky2 (Polygon, no trusted setup), STARK (StarkWare), or Halo2 (ECC, no toxic waste).

- Until the STARK migration is complete, publish the CRS verification instructions prominently and run a continuous public ceremony so participants can contribute at any time.

4. High-Severity Findings

HIGH-01 Intersection Attack Underestimated at Network Scale

• **HIGH** Location: Whitepaper §2, Spec №3 §8, §14

The protocol's traffic analysis resistance relies on constant-rate cover traffic and ZK-verified shuffling. However, it does not address the long-term intersection attack — a GPA observing which nodes are online across many epochs can statistically isolate senders even with perfect mixing.

The intersection attack (Berthold & Langos, 2002; Danezis, 2003) is distinct from timing correlation. It exploits the observation that across many message rounds, only nodes that are online during every round a sender is active can be the sender. Over time, the intersection of these sets shrinks toward a single node.

Phantom Protocol assumes that constant-rate cover traffic ('one packet per 100ms, always') prevents timing correlation. This is correct for single-round timing attacks. However, the intersection attack operates across rounds: a GPA who knows which Phantom nodes are online (observable from QUIC connection establishment and epoch registration in the DHT) can correlate message delivery with node uptime over days or weeks.

A 2024 paper, MixMatch (Oldenburg et al., PoPETs 2024:2), demonstrates that even with constant-rate cover traffic, flow matching attacks on mix networks remain effective when the adversary can observe ingress and egress with temporal resolution finer than the batch window. With a 200ms batch window and a GPA observing all QUIC connections, this temporal resolution is achievable.

The protocol's stated anonymity guarantee (' $1/k$ where k is the anonymity set size') refers to a single-epoch guarantee, not a multi-epoch guarantee. This distinction is absent from the whitepaper and specifications.

→ Recommendations

- Add explicit documentation distinguishing per-epoch anonymity from long-term anonymity. The current guarantee is per-epoch only.
- Implement variable node uptime patterns to disrupt intersection attacks: nodes should occasionally go offline and come back under different timing patterns, rather than maintaining constant presence.
- Consider a predecessor attack analysis (Wright et al., 2004) as a formal basis for the multi-epoch anonymity claim. This would bound the number of epochs before deanonymization becomes likely for a specific user.
- Evaluate PIR (Private Information Retrieval) techniques for DHT lookups to prevent the GPA from learning which nodes are querying which descriptors — this removes a significant intersection attack data source.
- The whitepaper should be updated to acknowledge that long-term anonymity is a separate property from per-epoch anonymity and that the current design does not provide formal long-term anonymity guarantees.

HIGH-02 DC-Net Jamming Attack Unaddressed

• **HIGH** Location: Whitepaper §9, Spec №1 §10

The DC-Net extreme mode has no mechanism to detect or exclude jamming participants. A single malicious group member can disrupt all DC-Net rounds indefinitely by broadcasting random bits, at zero cryptographic cost.

DC-Net security relies on honest participants XOR-combining their pads. A malicious participant who broadcasts random bits instead of their correct pad causes the XOR of all broadcasts to produce garbage, silently destroying the message without any observable difference in packet structure. This is known as the 'disruption' or 'jamming' attack and has been known since Chaum's original 1988 paper.

The Phantom specification acknowledges the collision resolution problem (two participants sending simultaneously) but does not address jamming. The 'reservation protocol' described in §9.2 of the whitepaper addresses intentional collisions but not malicious disruption.

Existing systems have addressed this: Dissent (Corrigan-Gibbs & Ford, 2010) uses verifiable shuffles to pre-assign transmission slots so jamming is attributable. Verdict (Ford et al.) uses ZK proofs over DC-Net ciphertexts to prove that a participant's broadcast is the correct XOR of their pads. These constructions are expensive but necessary for security.

At the Phantom DC-Net group size of 8–128 participants, a single malicious group member — whether a Sybil, a compromised node, or a state agent — can make DC-Net mode completely non-functional, forcing users to fall back to standard mode and giving up information-theoretic security.

→ **Recommendations**

- Implement Verdict-style verifiable DC-Net: each participant proves (via ZK) that their broadcast is the correct XOR of their shared pads with all group members. This makes jamming attributable and the jammer ejectable.
- Alternatively, adopt the Dissent approach: use a verifiable shuffle to pre-assign transmission slots before each round. This eliminates both jamming and unintentional collisions simultaneously.
- Add explicit DC-Net mode threat model documentation stating that the current (unprotected) design provides information-theoretic anonymity only against passive adversaries — it does not provide any guarantee when group members are malicious.
- Consider restricting DC-Net groups to nodes that have passed a higher admission threshold (e.g., higher uptime score, more epochs of proven ZK-shuffle compliance) to reduce the probability of a group containing a malicious node.

HIGH-03 Active Sybil Attack on DHT: Semantically Valid False Data

• **HIGH** Location: Spec №3 §4, §6, §8, §14

The 2025 IPFS/Kademlia active Sybil attack demonstrates that PoW-based Sybil resistance is insufficient when adversarial nodes respond with semantically valid but intentionally false data — a class of attack not addressed by Phantom-DPKI's admission tokens or multi-path lookup.

Phantom-DPKI's Sybil resistance relies on the dual PoW puzzle (C1=20 bits static, C2=16 bits dynamic). The multi-path lookup (d=5, quorum=3) is then used to detect eclipse attacks. The threat model assumes that false data is cryptographically detectable (fake descriptors fail signature verification).

A May 2025 paper from Inria (Netto, Cholez, Ignat) demonstrates a new class of attack on IPFS Kademia: adversarial nodes respond to DHT lookups with semantically correct and cryptographically valid data — specifically, valid provider records for a different content ID than the target. This attack stays below the detection threshold of statistical defenses while achieving 80% content eclipse on the latest IPFS release.

In Phantom's context, the analogous attack is: an adversary pre-mines valid NodeDescriptors (satisfying both PoW puzzles) for Sybil node IDs placed close to the XOR position of a target NodeDescriptor. When a client performs a lookup for the target node, the Sybils return valid descriptors for their own (adversary-controlled) nodes rather than the target. Since these descriptors pass all cryptographic checks, the multi-path quorum may converge on adversary-controlled nodes.

The 5-path quorum mitigates but does not eliminate this: if the adversary controls 3 of the 5 lookup paths (achievable with ~20% network penetration as Sybils near the target key), all 5 paths return valid but adversary-controlled descriptors, and quorum is satisfied with wrong data.

→ Recommendations

- Implement SR-DHT-Store (Inria, 2025): instead of publishing descriptors only to the k closest nodes, publish to a dynamically computed region of the keyspace using XOR distance estimation. This forces an attacker to pre-mine Sybils across a much larger keyspace region.
- Add a 'long-lived node preference' boost to lookup path selection: paths that pass through nodes with higher uptime_score and more epochs of observed compliance are preferred over paths through newer nodes, even if the newer nodes are XOR-closer to the target.
- Consider a DHT 'reputation region' mechanism: for each lookup, track which nodes provided correct responses historically and weight those nodes more heavily in future lookups for nearby keys.
- Add rate limiting on NodeDescriptor publication: a node that publishes a new descriptor more than once per epoch (except during the 120s overlap window) should be flagged and excluded from routing tables.

HIGH-04 QUIC Fingerprinting Bypasses 'Mimic HTTPS' Assumption

• **HIGH** Location: Spec №2 §3.1, Whitepaper §4, §13

Phantom Protocol relies on QUIC over UDP port 443 to mimic standard HTTPS traffic and evade traffic classification. Current ML-based DPI systems (2024–2025) can identify Phantom-style traffic despite this obfuscation, because the statistical properties of constant-rate encrypted traffic are distinct from real HTTPS.

The protocol states: 'Obfuscated to mimic standard HTTPS/QUIC traffic' as the transport layer property. This obfuscation relies on two properties: (1) all packets are fixed 1452 bytes, and (2) traffic is constant-rate. Both of these properties are, paradoxically, strong DPI fingerprints.

Real HTTPS/QUIC traffic has highly variable packet sizes and bursty timing patterns. A network with fixed-size packets at a constant 10-packet/second rate (one per 100ms cover slot) produces a traffic signature immediately distinguishable from any real HTTPS session by packet-size distribution analysis alone.

2024–2025 research confirms this threat is active. USENIX Security 2024 (Xue et al.) demonstrates fingerprinting of obfuscated proxy traffic by exploiting encapsulated TLS handshake structures. A 2025 NDSS paper identifies timing mismatches between OSI layers as protocol-agnostic fingerprints that bypass all existing obfuscation schemes. Chinese GFW has deployed ML-based QUIC analysis since April 2024 that decrypts QUIC Initial packets to inspect SNI fields.

Additionally, Phantom's packet size of exactly 1452 bytes is an unusual MTU choice (standard Ethernet MTU is 1500, typical QUIC payload after overhead is ~1200 bytes for standard applications). This specific value is a fingerprint in itself.

→ Recommendations

- Replace the fixed-size, constant-rate traffic model with an adaptive padding scheme that mimics realistic HTTPS traffic distributions. Consider FRONT (Gong & Wang, 2020) or NetShaper (differentially private network shaping, 2024) as the basis.
- Vary packet sizes using application-layer fragmentation and aggregation within Sphinx⁺ packet payloads: a single 1452-byte packet may carry fragments of multiple application messages, and conversely a large message may span multiple packets with genuine variable sizes at the IP layer.
- Implement a pluggable transport layer (analogous to Tor's obfs4 or Meek) that can be swapped without changing the core Sphinx⁺ format. This allows the transport obfuscation to be updated as DPI improves without a protocol version increment.
- The 1452-byte packet size should be re-evaluated. Randomizing between a small set of sizes (e.g., 500, 900, 1200, 1452 bytes) with application-layer padding makes statistical fingerprinting significantly harder.
- Consider domain-fronting or CDN-routing for the first hop to break the direct IP association between a client and the Phantom network, consistent with the 'no cleartext IDs' goal.

HIGH-05 Epoch Clock Skew: Expired Descriptor Injection Window

• **HIGH** Location: Spec №3 §10, Spec №1 §7, Spec №2 §3.2.3

The 120-second epoch overlap period combined with the ± 1 epoch acceptance window creates a 240-second window during which an adversary can inject or replay descriptors from the previous epoch, potentially routing traffic through stale or revoked nodes.

Spec №3 §10 specifies a 120-second overlap period during epoch transitions, and Spec №2 §3.2.3 specifies that nodes reject packets from epochs outside ± 1 of the current epoch.

Together, these create an acceptance window of up to 120 seconds on either side of an epoch boundary, giving a total injection window of 240 seconds.

An adversary who observes an ejection event during epoch N can keep routing traffic through the ejected node for up to 120 seconds into epoch N+1 by presenting the node's valid epoch-N descriptor (which still passes the ± 1 epoch check). During this window, the ejected node's misbehaviour is known but not yet fully enforced.

Additionally, the ± 1 epoch check applies to packet processing: a node in epoch N+1 will accept packets with epoch N. If an adversary has pre-recorded valid Sphinx⁺ packets from epoch N (possible because packet epochs are visible in plaintext in the packet header), they can replay these into epoch N+1 within the acceptance window. The replay cache keyed on `alpha_cl` prevents replay of the same packet, but a different valid packet from epoch N can still be forwarded.

The clock skew problem is exacerbated for nodes with imprecise clocks: nodes that are 60+ seconds behind will continue to accept epoch-N descriptors for up to 180 seconds into epoch N+1 in their local time.

→ Recommendations

- Tighten the epoch acceptance window to ± 30 seconds rather than ± 1 epoch (3600 seconds). This requires nodes to maintain well-synchronized clocks, which can be achieved through a network-internal time synchronization mechanism rather than relying on NTP.
- Add a monotonic epoch counter to the gossip ejection protocol: when a node is ejected, all peers record the ejection epoch and refuse to route through the ejected node for any epoch from the ejection epoch onward, regardless of the ± 1 epoch window.
- Remove the epoch field from the Sphinx⁺ packet header (it is currently plaintext) or encrypt it per-hop. A visible plaintext epoch enables statistical correlation across packets from the same epoch window.
- Implement NTS (Network Time Security, RFC 8915) for network-internal time synchronization to enforce clock consistency across nodes within ± 5 seconds.

5. Medium-Severity Findings

MED-01 Batch Timing Oracle: C_{in} Publication Timing Reveals Batch Membership

🟡 **MEDIUM** Location: Spec №2 §7, Figure 2

The requirement to publish C_{in} before packet processing — while cryptographically necessary — creates a timing oracle: a GPA can correlate the timing of C_{in} publications across the network to identify which batches are being processed simultaneously and which nodes are routing the same traffic.

The batch lifecycle requires C_{in} to be published to the DHT at T+200ms (after collection), C_{out} at T+400ms (after processing), and the proof π at T+700ms. An adversary monitoring DHT publications can identify all nodes that published C_{in} within the same 200ms window — these nodes are handling the same generation of packets.

By correlating C_{in} publication times with packet arrival times at known network observation points, a GPA can narrow the anonymity set to nodes that received packets in a specific time window. This is a form of intersection attack that operates at batch granularity rather than packet granularity.

The batch_{id} in the C_{batch} field (8 bytes) directly leaks which batch a packet belongs to if an adversary can observe the Sphinx⁺ packet headers. While the packet payload and routing are encrypted, the C_{batch} field is visible to any node that processes the packet.

→ Recommendations

- Randomize C_{in} publication timing within a ± 50 ms jitter window relative to the batch close time. This prevents temporal correlation of C_{in} publications without breaking the commitment-before-processing security requirement.
- Encrypt the C_{batch} field in the packet header on a per-hop basis using the hop's routing key, so only the current processing node can see the batch reference.
- Consider a two-phase commitment scheme: nodes commit to a hash-of-hash at T+200ms and reveal the full C_{in} hash only at T+400ms. This maintains the binding property while reducing the information available to a timing adversary.

MED-02 Proof-of-Work Difficulty: GPU/ASIC Scaling Not Accounted

🟡 **MEDIUM** Location: Spec №3 §4

The PoW puzzle difficulty (C1=20 bits static, C2=16 bits dynamic) is calibrated for CPU mining. A well-resourced adversary using GPUs can create Sybils approximately 100–1000x faster than the specification's estimates, reducing the effective deterrent significantly.

The specification estimates: 'Creating 1,000 identities: ~50 seconds, ~50 CPU-minutes' and 'Adversary with 1,000 cores creating Sybils: 14 CPU-hours / 1,000 = ~50 seconds.' This estimate assumes CPU-based BLAKE3 hashing at approximately 20M hashes/second/core.

BLAKE3 is specifically designed for high parallelism and performs extremely well on GPUs. A modern GPU (e.g., NVIDIA RTX 4090) achieves approximately 10–20 billion BLAKE3 hashes per second, compared to 200M hashes/second on a high-end CPU. A single GPU is therefore equivalent to approximately 50–100 CPU cores for this workload.

At 10 GH/s (modest GPU): $C_1=20$ bits ($2^{20} \approx 1\text{M}$ hashes) requires 0.0001 seconds per identity. Creating 1M Sybils takes 100 seconds on a single GPU. The dynamic puzzle ($C_2=16$ bits) adds 6.5ms per Sybil per epoch on a GPU — trivial even at scale.

The adaptive difficulty mechanism (C_2 increments when admission rate exceeds 10,000/epoch) may not respond fast enough to a GPU-based Sybil burst: in the one epoch before C_2 adjusts, millions of Sybil identities can be registered if an attacker uses a GPU farm.

→ Recommendations

- Replace or augment BLAKE3 with a memory-hard function (e.g., Argon2id or Scrypt) for the static puzzle. Memory-hardness significantly reduces GPU/ASIC advantage because GPU memory bandwidth does not scale as favourably as compute throughput.
- Alternatively, increase C_1 to 26–28 bits and use BLAKE3 in a tree-hashing mode that requires sequential dependencies, reducing parallelism. This would make the static puzzle 64–256x harder without changing the hash function.
- Add a burst detection mechanism: if more than 1,000 new node admissions are observed in any 10-minute window, immediately increment C_2 by 2 (rather than waiting until epoch end).
- Consider a proof-of-sequential-work component (e.g., VDF, verifiable delay function) for the dynamic puzzle. A VDF requires inherently sequential computation that cannot be parallelised — this directly counters GPU/ASIC scaling for the epoch-renewal cost.

MED-03 C_batch Truncation: 8-Byte Collision Space

🟡 MEDIUM Location: Spec №2 §3.2.7

The C_batch field in Sphinx⁺ packets is 8 bytes (64 bits), truncated from a 32-byte BLAKE3 digest. Birthday collision probability in a batch-dense network becomes non-negligible, potentially causing packets to be associated with wrong batch proofs.

The `C_batch` field is described as 'the first 8 bytes of a 32-byte Pedersen commitment hash.' It is used by verifiers to retrieve the full batch commitment and associated ZK proof from the DHT. With 8 bytes (64 bits), the birthday bound for collisions across all active batches is approximately 2^{32} batches — roughly 4 billion simultaneous active batches before a 50% collision probability.

For a single node processing one batch every 700ms, the number of batch IDs in circulation at any time is small. However, across a 100,000-node network each processing multiple batches per epoch (approximately 5,000 batches per node per epoch), the total number of active `C_batch` values is approximately 500 million per epoch. At this scale, the birthday probability of collision between any two batches within an epoch approaches 3%.

A collision between two batches' C_batch values means a verifier fetching the proof for batch A may retrieve the proof for batch B. This is not directly exploitable for deanonymization, but creates confusion in the verification pipeline and could be used to construct a DoS attack where an adversary deliberately creates batches whose C_batch values collide with legitimate nodes' batches.

→ Recommendations

- Increase C_batch to 16 bytes (128 bits). This is still small enough for the Sphinx⁺ header but pushes the birthday bound to 2^{64} , making collisions computationally infeasible even at network scale.
- Include the originating node_id in the C_batch derivation: C_batch = BLAKE3(commitment_hash || node_id)[:16]. This ensures C_batch values are globally unique even if two nodes happen to produce commitments with the same hash prefix.
- Update the packet format to accommodate the larger C_batch field — this requires a 8-byte header expansion, which can be accommodated by reducing the routing filler or reducing π_{ref} to 1 byte (batch sizes above 255 could use a separate lookup mechanism).

MED-04 Cover Traffic Model: Mandatory Rate Does Not Hide Application Behaviour

🟡 **MEDIUM** Location: Whitepaper §10, Spec №2 §3.2.2

Constant-rate cover traffic prevents packet-counting correlation but does not prevent application-behaviour fingerprinting: the pattern of when a node starts and stops having real traffic (even with constant cover) is itself a fingerprint.

The protocol mandates one packet per 100ms regardless of real traffic. This prevents an adversary from knowing whether any given 100ms slot contains real traffic. However, it does not prevent an adversary from observing that a node's 'real traffic' pattern (detectable via subtle timing variations in processing, or via application-level behaviour leaking through cover) correlates with specific communication patterns.

More critically, the cover traffic is generated by the Phantom daemon and is 'not configurable by the user.' This means the cover traffic rate is identical for all nodes — a strong fingerprint that a node is running Phantom. Real HTTPS traffic does not have exactly one packet per 100ms; it has bursty patterns. A node sending exactly 10 packets/second on UDP port 443, always, is immediately identifiable as running Phantom even without any payload inspection.

Additionally, loop-cover packets (routed through the mix network and returned) create a distinguishable circuit pattern: a node that always has circuits looping back to itself, at a consistent rate, with consistent path lengths, generates a recognizable traffic signature at intermediate nodes on the loop path.

→ Recommendations

- Implement Poisson-distributed inter-packet timing rather than fixed 100ms slots. A Poisson process with mean 100ms produces variable inter-packet intervals that are more consistent with real HTTPS traffic patterns.
- Add a configurable 'traffic shaping profile' that adapts the cover traffic rate and size distribution to match the ambient traffic distribution of the local network environment. This is analogous to Tor's current work on adaptive padding.
- Separate loop-cover from drop-cover in path selection: loop-cover packets should use full-length random paths (not shorter) and should not consistently return to the originating node — they should loop through the network and eventually be discarded, with only occasional return acknowledgements.
- Consider a FRONT-style defence: generate cover traffic that mimics the statistical distribution of the most common HTTPS application (e.g., web browsing traces), rather than a fixed-rate scheme.

MED-05 Hidden Service Introduction Point: PoW Difficulty Lag Creates DoS Window

● MEDIUM Location: Spec №1 §12

The introduction-layer PoW difficulty adjusts at epoch boundaries (every hour). A sustained DoS flood during an epoch can saturate introduction points for up to 3,600 seconds before difficulty increases, providing a full hour of effective service denial.

The adaptive PoW difficulty for introduction requests (default 18 bits, max 28 bits) is published in the ServiceDescriptor and adjusted per-epoch. If an adversary floods introduction points during an epoch, the difficulty does not increase until the next descriptor publication — up to 3,600 seconds away.

At 18 bits (262,144 hashes), an attacker with a GPU can generate approximately 38 million valid PoW solutions per second (10 GH/s / 262,144). At the default rate limit of 5,000 requests per minute, the attacker can saturate the introduction point's rate limit indefinitely while their PoW cost is negligible.

The introduction point's rate limit is also not dynamically adjustable — it is a fixed constant in the specification. An adversary who learns the MAX_INTRO_REQUESTS constant (which is published) can precisely calibrate their flood to stay just under the rate limit, maximising consumption of introduction point resources while avoiding triggering rate limit countermeasures.

→ **Recommendations**

- Implement intra-epoch difficulty adjustment: introduction points should track their own request rate and send an ephemeral updated difficulty signal (signed with their node key) to the service, which then publishes emergency descriptor updates. This allows mid-epoch difficulty adjustment within minutes rather than hours.
- Add IP-based rate limiting at the QUIC connection level, before PoW verification. This requires a QUIC connection per intro request, so IP-based rate limiting can reject connections from high-frequency sources before any PoW work is done.

- Implement a client puzzle that increases in difficulty based on the requester's recent request rate (using a short-term state table). First-time requesters get 18-bit puzzles; requesters seen 10 times in the last minute get 24-bit puzzles.
- Set MAX_INTRO_REQUESTS to be non-public and configurable by the service operator, rather than a hardcoded protocol constant.

MED-06 NodeDescriptor Size Creates DHT Bandwidth Pressure

🟡 **MEDIUM** Location: Spec №3 §5

The NodeDescriptor is 7.5KB uncompressed (dominated by the 3,293-byte Dilithium-3 signature). With $k=20$ replication and epoch rotation, the DHT bandwidth for descriptor maintenance scales to tens of megabytes per node per epoch — a significant burden on Light and Mobile nodes.

A full NodeDescriptor is approximately 7.5KB. With $k=20$ replication, each node's descriptor requires 150KB of DHT bandwidth per epoch for distribution. Across a 100,000-node network, the total DHT descriptor traffic is 15GB per epoch (4.2MB/s network-wide, sustained).

For a Light-tier node (10 Mbps uplink), the 150KB per epoch for its own descriptor is trivial. But Light nodes also store descriptors for up to $k=20$ nearby nodes (their k -bucket neighbors), and serve those descriptors to lookups. With $20 \text{ neighbors} \times 7.5\text{KB} = 150\text{KB}$ of local descriptor storage, plus serving these on demand, the Light-tier DHT overhead is manageable.

However, the 3,293-byte Dilithium-3 signature per descriptor is a real overhead that does not exist in classical-only systems. Dilithium-3 signatures are necessary for post-quantum security, but their size could be reduced by using Dilithium-2 (signature size 2,420 bytes) which targets NIST security level 2 (equivalent to AES-128 quantum security) rather than level 3 (AES-192). Given that the Ed25519 signature layer already provides classical security, the Dilithium layer needs only to provide post-quantum security against harvest-now-decrypt-later — level 2 is sufficient for this purpose.

→ Recommendations

- Evaluate downgrading from Dilithium-3 to Dilithium-2 for node identity signing. The Ed25519 layer provides classical security; the Dilithium layer needs only to prevent quantum key forgery. Dilithium-2 saves 873 bytes per signature (2,420 vs 3,293 bytes), reducing descriptor size from 7.5KB to 6.6KB.
- Implement descriptor compression using Zstandard (zstd) with a shared dictionary trained on typical NodeDescriptor content. The specification mentions '~4.5 KB average compressed' — enforce this and size the DHT bandwidth estimates on compressed sizes.
- Consider separating the signing key material from the per-epoch KEM keys into a smaller 'capability certificate' that is republished each epoch (containing only the epoch KEM keys and a short-lived signature) while the full descriptor with Dilithium-3 keys is published less frequently (e.g., once per day).

6. Low-Severity Findings

LOW-01 Replay Cache Memory Exhaustion

◦ **LOW** Location: Spec №2 §3.3 (step 3)

The replay cache keyed on alpha_cl (32 bytes per packet) is unbounded in the specification. Under a sustained packet flood, this cache grows without bound and can exhaust node memory.

Per-hop processing requires nodes to maintain a seen-packet cache to reject replays, keyed on the 32-byte alpha_cl field. The specification does not specify the cache expiry policy. If entries expire after one epoch (3600 seconds) and a node processes 10,000 packets per second at capacity, the cache requires $10,000 \times 3600 \times 32$ bytes ≈ 1.15 GB of memory — significant but manageable for Full-tier nodes.

However, an adversary sending crafted packets with random alpha_cl values (which all pass the initial version/epoch checks before the replay check) can fill the cache with entries that represent legitimate-looking but otherwise invalid packets. If the cache is not bounded, this causes OOM. If the cache is bounded with LRU eviction, legitimate replay protection can be bypassed by evicting recent entries.

→ Recommendations

- Implement a time-bucketed Bloom filter for the replay cache: use 2 Bloom filters, each covering half an epoch. At each half-epoch boundary, discard the older filter and start a new one. This bounds memory to $O(\text{packets_per_half_epoch})$ with configurable false positive rate.
- Set a maximum cache size (e.g., 500,000 entries) and implement a constant-time rejection mechanism for packets when the cache is full, rather than accepting them.
- Verify MAC (step 6 of Listing 1) before the replay check (step 3): invalid MAC packets are dropped immediately without cache insertion, eliminating the amplification vector.

LOW-02 mDNS Bootstrap Leaks Local Network Presence

◦ **LOW** Location: Spec №3 §11.1

The mDNS layer-1 bootstrap broadcasts the node's node_id on the local network. Any device on the same LAN can detect that a Phantom node is present and learn its node_id, potentially linking the Phantom identity to the local IP address.

mDNS announcements are broadcast to all devices on the local network segment. The specification states the announcement payload includes 'node_id (hex) + QUIC port.' This leaks the Phantom node's identity to any device on the LAN — including other network users, routers with logging enabled, and potentially malicious devices planted on shared networks (coffee shops, universities, corporate offices).

For a user in a hostile environment (e.g., a journalist in a country with aggressive surveillance), having Phantom's presence announced on the local network is a significant operational security

risk. The mDNS bootstrap is presented as the 'zero-external-dependency' path, but its privacy cost is high.

→ Recommendations

- Make mDNS bootstrap opt-in rather than enabled by default. Users in trusted home networks can enable it for faster bootstrap; users in hostile environments should default to DNS and genesis CID bootstrap only.
- Replace the plaintext node_id announcement with a challenge-response scheme: the node announces only a random nonce, and responds to queries with the node_id encrypted under a per-session key. This authenticates local peers without broadcasting the node_id in cleartext.
- Add a local network trust level to the configuration: TRUSTED (mDNS enabled), NEUTRAL (mDNS disabled, DNS and IPFS only), HOSTILE (all bootstrap via Tor-compatible pluggable transports only).

LOW-03 SURB Size Side-Channel

○ **LOW** Location: Spec №1 §8.3

The Single-Use Reply Block (SURB) has a fixed structure (32+96+128+32 = 288 bytes). An adversary observing IntroRequest sizes can distinguish SURB-carrying requests from non-SURB packets, potentially identifying hidden service connection attempts at the introduction point.

The IntroRequest message contains a SurbBlock of 288 bytes in addition to other fields. The total IntroRequest size is therefore a fixed, predictable value. If introduction points are observed by a network adversary, the packet size fingerprint of a SURB-carrying IntroRequest distinguishes it from other traffic arriving at the introduction point, even before decryption.

This is a narrow issue — the introduction point already knows a request is an IntroRequest by the fact that it is sent to the registration port — but the SURB size being constant and public aids traffic analysis of the traffic arriving at known introduction points.

→ Recommendations

- Pad IntroRequest messages to a fixed size using random padding, regardless of the actual SURB content size. This equalises the packet size fingerprint.
- Consider including a variable-length 'dummy SURB' in packets that do not carry real SURBs, making the presence of a SURB indistinguishable from its absence at the size level.

LOW-04 Epoch-0 KEM Keys Permanently Embedded in Service Address

◦ **LOW** Location: Spec №1 §3.1

The .phantom address derivation includes the epoch-0 X25519 and Kyber-1024 public keys. These keys are permanently bound to the address and cannot be rotated, creating a long-term forward secrecy gap for the address-binding property.

The service address is computed as `BLAKE3('phantom-hs-v1' || pk_ed || pk_dil || pk_x25519_epoch0 || pk_kyber_epoch0)`. The epoch-0 KEM keys are included to ensure the address is self-certifying without requiring a DHT lookup. However, these epoch-0 keys are permanent — they define the address — and their corresponding private keys must be retained indefinitely.

If an adversary compromises the epoch-0 KEM private keys (e.g., through the quantum-era recovery of classical X25519 private keys), they can derive the address preimage and produce fraudulent service descriptors that the address self-authentication check will reject... but the address derivation itself may no longer be assumed to be collision-resistant against quantum preimage attacks, since `pk_x25519_epoch0` and `pk_kyber_epoch0` are classical and post-quantum respectively, but the address is a hash of their concatenation.

The more immediate concern is operational: the specification does not address what happens to epoch-0 private keys after the service has been running for a year. If they are kept for address authentication purposes, they become long-term secrets that must be protected indefinitely.

→ **Recommendations**

- Separate address derivation from KEM key derivation: derive the address only from the permanent signing keys (`pk_ed || pk_dil`), which are already kept long-term. Remove the epoch-0 KEM keys from the address derivation.
- If self-authentication requires KEM keys in the address, use a commitment to the epoch-0 KEM keys (`BLAKE3(pk_x25519_epoch0 || pk_kyber_epoch0)`) rather than the keys themselves, and include this commitment in the address. Clients retrieve the full epoch-0 KEM keys from the first descriptor and verify the commitment.
- Add explicit key custody documentation to Spec №1 specifying the lifecycle of epoch-0 KEM keys and their secure deletion timeline.

7. Informational Findings

INFO-01 No Formal Adversary Capability Document

i INFO Location: Whitepaper §2

The threat model in the whitepaper is described informally. A formal adversary specification (in the Universal Composability or game-based security model) is absent, making it difficult to assess whether the protocol's security properties are composable and whether the claimed guarantees hold in the presence of concurrent protocol instances.

The whitepaper defines GPA capabilities in prose but does not specify: (1) whether the adversary is static or adaptive; (2) whether the adversary is computationally bounded or information-theoretically unbounded; (3) how the adversary's capabilities compose with the ZK shuffle proof system (which has a game-based security definition) and the DC-Net (which is information-theoretically secure). Without a formal model, protocol compositions may introduce unintended interactions.

→ Recommendations

- Produce a formal security definition document, ideally in the Universal Composability (UC) framework, defining the ideal functionality F_{Phantom} that the protocol is designed to realise. This document should be a prerequisite for the Phase 5 security audit.
- At minimum, specify whether the anonymity guarantee is computational or information-theoretic for each protocol mode (standard mix mode vs. DC-Net mode) and against which adversary class.

INFO-02 No Node Operator Incentive Mechanism

i INFO Location: Whitepaper §12

Phantom Protocol has no economic incentive for node operators. Without incentives, the network may fail to attract and retain Full-tier nodes, leading to a routing table dominated by Mobile and Light nodes that cannot generate ZK proofs.

Full-tier nodes (8-core CPU, 100 Mbps uplink, proof generation capability) are required for verifiable mixing. The protocol currently has no mechanism to reward node operators. In contrast, Nym provides token-based incentives and Tor relies on volunteer altruism. At scale, altruism alone is insufficient to maintain the Full-tier node density required for the ZK shuffle proof to function network-wide.

The specification is aware of this but defers it to future design. However, incentive design decisions significantly affect the protocol's security: incentive mechanisms can create centralisation pressure (as with Nym's token economics) or gaming opportunities (if reputation scores can be manipulated for advantage).

→ **Recommendations**

- Prioritise the incentive design problem to Phase 3, not after Phase 6 (ecosystem). Without a viable incentive model, the Phase 5 public testnet will not attract sufficient Full-tier operators to test the network at realistic scale.
- Evaluate non-token incentive mechanisms: reciprocal routing (nodes that route more traffic receive more routing capacity in return), reputation-based preferential circuit placement, and bandwidth credit systems that do not require a separate token.

INFO-03 Genesis CID Upgrade Path Undefined**i INFO** Location: Spec №3 §11.3, §15

The hardcoded genesis CID creates a bootstrap dependency on a specific IPFS content address. The process for distributing an updated genesis CID in the event of a major protocol upgrade is not specified.

The spec acknowledges: 'A bootstrapless upgrade mechanism (propagated via the live network) is a future design goal.' Until this mechanism exists, protocol upgrades require a new genesis CID distributed via software update — effectively a centralised distribution mechanism that is antithetical to the protocol's 'no centre' principle.

→ **Recommendations**

- Design a genesis CID self-update mechanism before Phase 1 ends: the current genesis CID should include metadata specifying a cryptographically signed upgrade announcement location. Nodes periodically check this location (via DHT, not a central server) for newer genesis CIDs signed by the protocol upgrade committee.
- Define the 'protocol upgrade committee' governance model explicitly. Without this, the genesis CID is effectively controlled by whoever holds the signing keys — a significant centralisation risk.

8. Priority Remediation Roadmap

The following table orders all findings by recommended implementation priority, accounting for both severity and implementation dependency.

Priority	Finding	Severity	Phase to Address	Effort
1	CRIT-02: BLS12-381 CRS ceremony planning	CRITICAL	Before Phase 2	High — requires MPC coordination
2	CRIT-01: Kyber fragment scheme cryptographic review	CRITICAL	Phase 1 (parallel)	High — requires external cryptographer
3	HIGH-03: Active Sybil / SR-DHT-Store	HIGH	Phase 1	Medium — DHT publication strategy
4	HIGH-05: Epoch clock skew / epoch field encryption	HIGH	Phase 1	Low-Medium — spec change
5	HIGH-04: QUIC fingerprinting / pluggable transport	HIGH	Phase 3	High — core transport redesign
6	HIGH-01: Intersection attack documentation and mitigation	HIGH	Phase 2	Medium — design + documentation
7	HIGH-02: DC-Net jamming / verifiable DC-Net	HIGH	Phase 4	High — new ZK component
8	MED-02: PoW memory-hardening	MEDIUM	Phase 1	Low — replace BLAKE3 for static puzzle
9	MED-03: C_batch field expansion to 16 bytes	MEDIUM	Phase 1	Low — format change
10	MED-01: Batch timing jitter for C_in publication	MEDIUM	Phase 2	Low — timing randomisation
11	MED-04: Cover traffic Poisson model	MEDIUM	Phase 3	Medium — timing model change
12	MED-05: Intro DoS intra-epoch difficulty adjustment	MEDIUM	Phase 4	Medium — signalling protocol
13	MED-06: Descriptor size / Dilithium-2 evaluation	MEDIUM	Phase 1	Low — parameter choice
14	LOW-01: Replay cache Bloom filter	LOW	Phase 2	Low — implementation detail
15	LOW-04: Epoch-0 KEM keys in address	LOW	Phase 1 (spec change)	Low — spec change before implementation
16	LOW-02: mDNS opt-in	LOW	Phase 1	Low — config flag
17	LOW-03: SURB padding	LOW	Phase 4	Low — padding addition
18	INFO-02: Incentive mechanism design	INFO	Phase 3	Very High — open research problem

Priority	Finding	Severity	Phase to Address	Effort
19	INFO-01: Formal security model	INFO	Phase 5 (prerequisite)	High — research task
20	INFO-03: Genesis CID upgrade path	INFO	Phase 1	Medium — governance design

Table 2 — Priority remediation roadmap

9. Positive Findings

This audit identified several areas where the Phantom Protocol specification demonstrates strong security design that exceeds the state of the art:

- **Self-certifying identities:** The `node_id = H(pubkeys)` construction for both relay nodes and hidden services is elegant and removes all PKI trust assumptions. No known attack applies to this construction with BLAKE3.
- **ZK shuffle proof concept:** The application of Bayer-Groth verifiable shuffle to the mix network layer is the most significant innovation in the protocol and addresses the fundamental weakness of all existing deployed mix networks. If the CRS ceremony is properly conducted, this construction provides strong soundness guarantees.
- **Dual-layer post-quantum construction:** The hybrid X25519+Kyber-1024 approach (with BLAKE3 combiner) correctly follows the industry consensus for hybrid key exchange. The combiner construction is secure if either primitive is secure — this is the correct design.
- **Gossip ejection protocol:** The evidence-based ejection protocol with verifiable evidence requirements is well-designed. The requirement that ejection notices include cryptographically verifiable evidence (not just accusations) prevents false ejection attacks effectively.
- **Epoch-based key rotation:** One-hour epoch rotation of KEM keys provides meaningful forward secrecy. The separate permanent/ephemeral key distinction (signing keys permanent, KEM keys rotating) correctly balances identity stability with session secrecy.
- **Multi-path eclipse-resistant lookup:** The $d=5$ disjoint path lookup with quorum=3 is the correct application of S/Kademlia's eclipse resistance analysis. The 0.0001% eclipse probability at 20% adversarial nodes is consistent with the cited S/Kademlia paper.
- **Commitment-before-processing rule:** The requirement to publish `C_in` before any packet processing is a critical security invariant correctly identified and enforced. Its DHT timestamp provides proof-of-binding that prevents adaptive commitment selection.
- **AS-diversity for introduction point selection:** Requiring introduction points to span multiple autonomous systems is a sound operational security practice that limits the impact of ISP-level adversaries on hidden service availability.

10. References

- [OFX24] Nym Technologies — Outfox: A Post-Quantum Packet Format for Layered Mixnets. arXiv:2412.19937, December 2024.
- [KYB24] Bernstein et al. — KyberSlash: Exploiting Secret-Dependent Division Timings in Kyber Implementations. IACR ePrint 2024/1049.
- [BER26] Berzati, Viera, Chartouny, Vigilant — Simple Power Analysis Assisted Chosen Cipher-Text Attack on ML-KEM. CASCAD 2025 / Springer LNCS 15952.
- [INR25] Netto, Cholez, Ignat — Active Sybil Attack and Efficient Defense Strategy in IPFS DHT. Inria hal-05424411, May 2025.
- [ACM24] Netto, Cholez — Sybil Attack Strikes Again: Denying Content Access in IPFS. ARES 2024, ACM, July 2024.
- [MIX24] Oldenburg, Juarez, Rúa, Diaz — MixMatch: Flow Matching for Mixnet Traffic. Proceedings on Privacy Enhancing Technologies 2024(2):276–294.
- [UNI24] Xue, Kallitsis, Houmansadr, Ensafi — Fingerprinting Obfuscated Proxy Traffic with Encapsulated TLS Handshakes. USENIX Security 2024.
- [NDN25] Cross-layer RTT Fingerprinting of Proxy Traffic. NDSS 2025.
- [HOT25] Lavrentieva, Juarez, Honda — Rethinking the Role of Network Stacks for Website Fingerprinting Defenses. HotNets 2025.
- [GFW24] China GFW QUIC Censorship — SNI inspection of QUIC Initial packets. Reported April 2024.
- [BG12] Bayer, Groth — Efficient Zero-Knowledge Argument for Correctness of a Shuffle. EUROCRYPT 2012.
- [DAN03] Danezis — Statistical Disclosure Attacks. IFIP SEC 2003.
- [WRI04] Wright, Adler, Levine, Shields — The Predecessor Attack: An Analysis of a Threat to Anonymous Communications Systems. ACM TISSEC 2004.
- [A16Z] a16z crypto — On-Chain Trusted Setup Ceremony. 2025. (BLS12-381 CRS analysis)
- [GRO16] Groth — On the Size of Pairing-Based Non-Interactive Arguments. EUROCRYPT 2016. (Groth16 ZK system)
- [CHE15] Cheon — Security Analysis of the Strong Diffie-Hellman Problem. EUROCRYPT 2006. (Cheon attack on trusted setups)
- [COR10] Corrigan-Gibbs, Ford — Dissent: Accountable Anonymous Group Messaging. ACM CCS 2010.
- [BM07] Baumgart, Mies — S/Kademlia: A Practicable Approach Towards Secure Key-Based Routing. IEEE ICPADS 2007.
- [NIST203] NIST — ML-KEM (CRYSTALS-Kyber), FIPS 203. August 2024.
- [NIST204] NIST — ML-DSA (CRYSTALS-Dilithium), FIPS 204. August 2024.

— Phantom Protocol Security Architecture Audit — End of Report —